

Formalization workflow for U-AUB-02

Kieran McShane

2026-06-07

Formalization workflow for U-AUB-02

Source of truth

The Lean source remains the authority. Article prose, blueprint statements, and ticket reports must be generated from checked declarations, board entries, and axiom audits.

Current main module:

`PptFactorization.AppendixBAubrunGraduate`

Global status:

U-AUB-02 is open. The active endpoint is a checked conditional reduction, not a no-input Aubrun count theorem.

Active theorem-facing endpoint:

`AppendixB.AubrunBiDefectCountBound_of_bianeAdjacentLinkHalves_left_gap_three_start_impossibl`

This is the current reduction layer to track while U-AUB-02 remains open. It is sharper than the previous primitive-child frontier because the five strict smaller child primitive leaves are now supplied internally by the parent primitive residual input. The remaining smaller nonwrapping child leaf is the genuine internal-subinterval collision branch.

Previous reduction layer:

`AppendixB.AubrunBiDefectCountBound_of_bianeAdjacentLinkHalves_left_gap_three_start_impossibl`

Delta: the previous layer still exposed internal S , start S , end S , fixed predecessor, adjacent T , and genuine shorter internal-subinterval as separate smaller child leaves. The active layer uses `leftTightStrictReturnIntervalPrimitiveResidual_of_strictChildPrimitive` to absorb the five primitive alternatives into the parent primitive leaf.

Current theorem-facing surface:

Every item in the following list is still an input to the active endpoint, not a discharged subgoal. All finite gap-three branches are omitted because they are

now supplied inside the endpoint.

large-gap left Biane link, only for $2 < c-a$ and $c-a \neq 3$
large-gap right Biane link, outside the checked $d-b=3$, $c=b+2$ slice
five unchanged side-specific residual branch collision leaves
one smaller nonwrapping child internal-subinterval collision leaf
Chapuy/trisection recurrence

Presentation rule: do not cite the active endpoint as “the Aubrun theorem” without the qualifier “conditional reduction from the remaining theorem-facing leaves”. The name contains many checked components, including `chapuy`, but Chapuy is still a hypothesis of the endpoint, not an internal proof.

The left-start slice $c-a=3$, $b=a+1$ no longer appears on this surface. Its four former branches are discharged internally: one from the right frontier, two by forcing that right-frontier contradiction, and the residual `not pi(a,a+2)` and `S(a,a+2)` branch by tree uniqueness for the copied length-two contour path.

Scientific toolchain

The local proof workflow now has a single status command:

```
python3 tools/scientific_tool_status.py
```

It checks Python, Lake, Graphviz, Pandoc, latexdiff, leanblueprint, the leanblueprint plasTeX runtime, Grok, optional narration availability, the reference-library bridge, and the blueprint source files. The command reports readiness only; it does not print secrets.

Finite permutation exploration is handled by:

```
python3 scripts/u_aub_02_perm_explorer.py --max-n 7  
python3 scripts/aubrun_branch_sanity.py --max-n 9 --all
```

These scripts are conjecture-finding and regression tooling only. Their outputs cannot close a Lean theorem-facing leaf.

The proof-frontier picture is generated by:

```
python3 tools/u_aub_02_frontier_graph.py
```

It writes `build/u_aub_02_frontier.{dot,svg,png}` and deliberately marks the four live leaves as open.

The maintained artifact build is:

```
tools/build_formalization_artifacts.sh
```

This rebuilds the status graph, blueprint PDF/web output, blueprint declaration checks, and the Pandoc/LaTeX exposition PDFs. The script pins the leanblueprint plasTeX runtime; using the global `plastex` binary can silently omit the leanblueprint plugins.

Zotero is available through:

```
tools/zotero_project_refs.sh status
tools/zotero_project_refs.sh search "Aubrun"
tools/zotero_project_refs.sh export-all
```

The reference bridge is optional. Bibliography writes or imports should still be treated as explicit project changes.

Sage/GAP is optional. Use it for larger finite-combinatorics experiments if it is available:

```
brew install --cask sage
```

Checked normalization machinery for the current gap-three surface:

```
AppendixB.AubrunBiDefectCountBound_of_bianeAdjacentLinkHalves_left_gap_three_start_impossible
AppendixB.AubrunBiDefectCountBound_of_bianeAdjacentLinkHalves_left_gap_three_start_impossible
AppendixB.leftTight_bianeAdjacentLink_left_of_gap_three_start_adjacent_reduced_fixed_a2_no_pi
AppendixB.leftTight_bianeAdjacentLink_left_of_gap_three_start_adjacent_target_impossible
AppendixB.leftTight_bianeAdjacentLink_left_of_gap_three_start_adjacent_branch_to_target_iff
AppendixB.leftTight_bianeAdjacentLink_left_of_gap_three_start_S_b_c_false_of_global_right_fix
AppendixB.leftTight_bianeAdjacentLink_left_of_gap_three_start_fixed_a2_false_of_global_right_fix
AppendixB.leftTight_bianeAdjacentLink_left_of_gap_three_start_T_b_a2_false_of_global_right_fix
AppendixB.leftTight_bianeAdjacentLink_left_of_gap_three_start_S_a_a2_no_pi_false
AppendixB.leftTight_bianeAdjacentLink_right_of_gap_three_end_adjacent_of_pi_b_c
AppendixB.leftTight_bianeAdjacentLink_right_of_gap_three_end_adjacent_of_S_b_c
AppendixB.leftTight_bianeAdjacentLink_right_of_gap_three_end_adjacent_T_b1_c_to_S_b1_d_of_right
AppendixB.leftTight_bianeAdjacentLink_right_of_gap_three_end_adjacent_fixed_b1_to_S_b1_d
AppendixB.leftTight_right_gap_three_T_b1_c_of_S_b1_d_of_not_S_c_d
AppendixB.leftTight_bianeAdjacentLink_right_of_gap_three_end_adjacent_of_S_b1_d_of_left_shift
AppendixB.leftTight_bianeAdjacentLink_right_of_gap_three_end_adjacent_reduced_fixed_b1_no_pi
AppendixB.leftTight_bianeAdjacentLink_left_of_gap_three_end_adjacent_reduced
AppendixB.leftTight_bianeAdjacentLink_left_of_gap_three_end_adjacent_of_pi_a_a1
AppendixB.leftTight_bianeAdjacentLink_left_of_gap_three_end_adjacent_reduced_no_pi_a_a1
AppendixB.leftTight_bianeAdjacentLink_left_of_gap_three_end_adjacent_of_pi_a1_b
AppendixB.leftTight_bianeAdjacentLink_left_of_gap_three_end_adjacent_reduced_no_pi_branches
AppendixB.leftTight_bianeAdjacentLink_left_of_gap_three_end_adjacent_reduced_fixed_a1_no_pi
```

These helpers justify branch rewrites inside the active endpoint. Their checked status is not an unconditional Aubrun count theorem.

The checked right gap-three reducer consumes the branch $\pi(b, c)$ by transitivity with the crossing link $\pi(a, c)$. The adjacent-repeat branch $S(b+1, c)$ has been normalized to the fixed-point branch $\pi(b+1)=b+1$, and the special branch $S(b, c)$ is now closed by copying the contour step $c \rightarrow d$ across $S(b)=S(c)$ and comparing it with the direct $\pi(b, d)$ path. The branch $\pi(b+1, c)$ is now routed through the shifted crossing $(a, b, b+1, d)$: it gives $\pi(a, b+1)$, the large-gap right input gives $S(b+1, d)$. The fixed branch $\pi(b+1)=b+1$ is routed through the same branch: fixedness gives $S(b+1, c)$, and denying $S(b+1, d)$ creates a

length-four incidence path from $S(b)$ to $S(d)$ that contradicts the direct length-two path from $\pi(b,d)$. The last branch $S(b+1,d)$ is now closed: if $S(c,d)$ failed, incidence-path uniqueness forces $\pi(b+1,c)$; then the shifted left link gives $S(a,b)$, and the incidence-square obstruction contradicts the branch. The active endpoint signature therefore contains no `hRightS_b_c` or `hRightT_b1_c` or `hRightFixed_b1` or `hRightS_b1_d` parameter. The left start-adjacent slice $c-a=3$, $b=a+1$ is fully discharged at the active endpoint. The left-start $S(b,c)$ usage in that now-discharged slice is supplied through the global right-frontier wrapper; this is distinct from the right gap-three special branch $S(b,c)$ above, which is now discharged locally. The fixed $a+2$ branch and the $\pi(b,a+2)$ branch both force the left-start $S(b,c)$ contradiction. The residual `not  $\pi(a,a+2)$` and $S(a,a+2)$ branch is closed by copying the contour step from $a+2$ to c into a length-two incidence path from $S(a)$ to $S(c)$; tree uniqueness compares it with the direct path from $\pi(a,c)$ and forces the forbidden link $\pi(a,a+2)$. The left gap-three end-adjacent slice $c-a=3$, $b=a+2$ is reduced to two finite branches. The branch $\pi(a,a+1)$ is absorbed by the shifted gap-two left theorem. The branch $\pi(a+1,b)$ is absorbed by the already visible start-adjacent branch on the shifted crossing $(a,a+1,c,d)$. The adjacent-repeat branch $S(a+1,b)$ is normalized to the fixed-point branch $\pi(a+1)=a+1$; the other theorem-facing end-adjacent branch left in that slice is $S(a+1,c)$.

These branch counts are endpoint-local snapshots, not a progress percentage. Every finite-branch removal must still restate the non-local leaves separately: the large-gap Biane inputs, the nested non-direct return-interval collision theorem, and the Chapuy/trisection recurrence. A wrapper only counts as real progress when the named endpoint exposes strictly fewer branch hypotheses after the change.

Verification tiers

Use three tiers, in this order:

1. micro-check a local theorem or signature with `lake env lean`;
2. quick-check the enclosing module with `scripts/lean_quick_check.py`;
3. rebuild the enclosing module with `lake build`.

Only after the module build passes should the ledger, board, article note, or blueprint be updated.

Branch-probe discipline

The Appendix B file is large and declaration order matters. New Biane finite branch lemmas should therefore be developed outside the endpoint first:

1. test the proposed branch implication in small concrete models;
2. prove the helper in a downstream Lean probe importing `PptFactorization.AppendixBAubrunGraduate`;
3. only then move the checked helper into the earlier local branch cluster, so the public endpoint never references a declaration that appears later in

the file.

The finite sanity command is:

```
python3 scripts/aubrun_branch_sanity.py --max-n 9
```

This script enumerates small left-tight permutations and searches for counterexamples to the currently exposed branch implications. Since U-AUB-322 closed the finite gap-three branch list, it now also reports a live large-gap profile. Through `n=9`, the profile checks 6,916 left-tight permutations and finds no crossing hypotheses at all. This is not proof evidence: it supports the expected direct noncrossing/large-gap route, but it does not close the structural Biane theorem. Its purpose is only to stop obviously false local branch attempts before they enter Lean.

Article track

Narrate Lean is the preferred first pass:

```
uvx narratelean narrate PptFactorization/AppendixBAubrunGraduate.lean \
  --no-definitions --no-proofs \
  -o exposition/narratelean_appendix_b_statements.md
```

Current policy: narration is optional exposition support, not proof evidence. Until a narration pass produces useful mathematics, leanblueprint plus hand-maintained Pandoc notes are the reliable path. The note is written directly from theorem statements, proof-artifact reports, and blueprint declaration checks. Any future Narrate Lean output still has to be audited against the same distinction: checked reduction is not no-input closure, and local branch normalization is not a theorem-strength leaf closure.

Pandoc is the Markdown-to-LaTeX bridge:

```
pandoc exposition/u_aub_02_formalization_note.md \
  -o exposition/u_aub_02_formalization_note.tex
```

Blueprint track

Leanblueprint has been initialized in `blueprint/`.

Use these conventions:

1. Put mathematical prose in `blueprint/src/content.tex`.
2. Attach Lean declarations using `\lean{...}`.
3. Mark statements with `\leanok` only after the declaration builds and the axiom audit is benign.
4. Keep open theorem-strength leaves visibly unmarked or marked as open in prose.
5. For conditional reduction theorems, say explicitly that `\leanok` verifies the reduction from its hypotheses, not the hypotheses themselves.

6. Every open leaf should name the missing invariant it is expected to prove or preserve, not only the Lean placeholder name.

Useful commands once content is stable:

```
CFLAGS="-I/opt/homebrew/include" LDFLAGS="-L/opt/homebrew/lib" \
uvx leanblueprint pdf
```

Then generate the web declaration cache:

```
CFLAGS="-I/opt/homebrew/include" LDFLAGS="-L/opt/homebrew/lib" \
uvx leanblueprint web
```

Finally check that every `\lean{...}` declaration referenced from the blueprint exists:

```
CFLAGS="-I/opt/homebrew/include" LDFLAGS="-L/opt/homebrew/lib" \
uvx leanblueprint checkdecls
```

In this repo, `lake build EnsX2026` may be needed once before `checkdecls`, because the Lake project still advertises that auxiliary library.

Doc generation through `doc-gen4` is deferred. The first attempt tried to switch to `leanprover/lean4:v4.31.0-rc1` and download a large toolchain. Do not enable that until the project deliberately chooses a documentation toolchain policy.

LeanArchitect track

LeanArchitect is the right long-term model: keep blueprint metadata close to Lean declarations, infer dependencies, and export synchronized project documents. It is not currently installed in this local repo, so the immediate workflow is:

1. maintain leanblueprint declarations now;
2. keep each blueprint theorem linked to exactly one or a small number of Lean constants;
3. later migrate those links to LeanArchitect annotations if the package is added to the project.

Adversarial review loop

After any significant proof-frontier or exposition change:

1. ask Grok for adversarial feedback on both the article-style exposition and the workflow note;
2. treat the feedback as advisory, not as proof evidence;
3. patch only claims that can be verified against Lean declarations, build output, axiom audits, or generated artifacts;
4. rerun Pandoc and leanblueprint checks after incorporating accepted changes.

Use the project review command:

```
tools/grok_review_u_aub_02.sh
```

It asks for adversarial feedback from the current theorem-frontier context and keeps the retained Markdown review at `build/grok_u_aub_02_review.md` mathematical.

Durable summaries of useful Grok feedback should be tracked rather than left only under `build/`. The current summary is the Grok workflow review note in the exposition folder. Its main guardrail is to foreground the four open theorem-strength leaves before the long checked endpoint name, because a checked conditional adapter can otherwise feel psychologically close to a done ticket.

Before updating this workflow note or the article note, print the current theorem header of the sharp `AubrunBiDefectCountBound_of_*` declaration and confirm that the listed leaves match its parameters: large-gap left outside `c-a=3`, large-gap right outside `d-b=3` with `c=b+2`, no finite gap-three branch parameters, the five unchanged side-specific residual collision inputs, the smaller nonwrapping child internal-subinterval input, and `hstep`.

The recurring review questions are:

1. Does the exposition overclaim a conditional reduction as an unconditional theorem?
2. Are all theorem-facing leaves named in plain mathematical terms?
3. Does every `\leanok` marker correspond to a declaration that builds and has a benign axiom audit?
4. Are the remaining invariants structural, or is the text merely listing cases without a proof strategy?
5. Does the endpoint-local branch ledger strictly shrink, while the hard collision and Chapuy leaves remain visible when they are still open?

Exit criteria for U-AUB-02

Do not mark U-AUB-02 done until the public Aubrun expectation pipeline no longer exposes the corrected count theorem as an input. At minimum, Lean must check:

1. large-gap left Biane adjacent link outside `c-a=3`;
2. large-gap right Biane adjacent link;
3. nested non-direct return-interval collision theorem;
4. Chapuy/trisection recurrence;
5. the final bidefect count endpoint;
6. the public expectation pipeline consuming that endpoint.

Each public endpoint must have an axiom audit no worse than:

```
[proptext, Classical.choice, Quot.sound]
```